

COPILOTTRAINING
TECHNICAL BRIEFING

Local Agentic SDLC

A Loop with a Contract

Roles own the work. Tests decide done. Budgets decide when to stop. A person decides what merges.

Agents at Your Keyboard · CopilotTraining Tech Talk · 45 min

🤔 THE CORE QUESTION

"How can a developer run plan, implement, test, review, and integrate locally with custom agents, and know when the ru…

An orchestrator owns the sequence; each subagent owns one role.

Write the loop contract first, then let evidence steer the run.

Developer starting the loop

Writes the contract, launches sessions, and reads the evidence

Reviewer of the draft

Receives an integrated pull request and owns the merge decision

Team lead

Owns the budget and escalation policy every run inherits

2 issues → 1 draft PR

FanHub #4 and #5 integrated into draft #217 (observed)

96 / 96 tests

Integrated suite after a reviewer-driven fix (observed)

≈380 AI credits

One observed run across four sessions; no manual baseline

TABLE OF CONTENTS

PART 1

Write the Contract

Roles, done, budget, merge owner

A plan with risks before code and a split test with named seams

PART 2

Build the Crew

Orchestrator plus four role agents

Boundaries that held, one worktree per workstream

PART 3

Evidence Decides

Tests steer; the Stop hook backs them up

Predicted risk, routed failure, staged backstop, budget stop

PART 4

Hand It Off

Integration record and draft PR

A reviewer catch after merge, and what the reviewer receives

Four CLI Sessions Took Two Issues to One Integrated Draft

FanHub, 2026-10-22, Copilot CLI 1.0.100 · REAL run · AI credits per session

Plan	Plans for #4 and #5 posted on the issues; two workstreams; three named seams	47.15
Implement #4	Worktree char-detail · bee5680 · 56/56 · reviewer accept after one round	104.12
Implement #5	Parallel worktree episode-detail · 24b6a6c · 8 failed → 47/47	102.55
Integrate	Seams resolved · reviewer catch after merge · 96/96 · draft #217	126.13

One observed run: about 380 credits; developer time and a manual baseline were unmeasured. What made it trustworthy?

PART 1

Write the Loop Contract First

Four answers before the first run, a plan with risks before code, and a split test with named seams



Four fields

Role ownership, completion evidence, budget, merge authority



Risks before code

The plan names what could break and how to avoid it



Split test

Disjoint files apart from named seams, each owned

```
copilot --agent sdlc-orchestrator --mode autopilot -p "plan #4 #5"
↳ two plans posted as issue comments · split into two workstreams
```

Four Answers Turn a Hand-Off Into Checks

The orchestrator states the contract in its first message and its final summary

Who does what?	planner plans, implementer edits source, test-writer edits tests, test-reviewer judges	`agent tools + body`
When is it done?	Frontend Jest suite passes and the reviewer accepts	`Stop hook + reviewer`
When does it give up?	Autopilot continues, AI-credit cap, three blocked stops	`CLI flags + hook`
Who decides what merges?	A person reviewing a draft pull request	`deny git push · draft PR`

Each field names where it is enforced, so each one can be checked from the run's own artifacts

The Planner Names the Risk Before Anyone Writes Code

planner output shape · posted as an issue comment

```
## Plan for #5: Episode detail page
**Goal:** one sentence a reviewer can check
**Files to change:** path, what changes; (seam) marked
**Files to create:** path, purpose
**Risks before code:** Episodes.test.js renders the
  page without a router; navigation will break it
**Tests to add:** file, behaviors to prove
**Done when:** observable checks + suite command
```

🔮 #5 risk: missing router context

Named in the plan; it surfaced as 8 failing tests in Part 3

🏛️ #4 risk: class component

Characters.jsx needs Link wrappers in place of hooks

💬 Plan lives on the issue

gh issue comment puts it where the team already reviews work

Split When Files Are Disjoint Apart From Named Seams

✂ Split

Two workstreams

- File sets disjoint apart from named seams
- Each workstream passes the suite alone
- FanHub: #4 and #5 split

🔗 Keep together

One workstream

- Both plans edit the same component
- One plan depends on the other's output
- Integration cost exceeds parallel gain

🔗 Name every seam

Owned by integration

- App.js: both add a route
- App.test.js: both add route tests
- node/README.md: both document

💡 Related entities remain noninteractive: that one plan line let each page ship without the other.

PART 2

Build the Orchestrator and Its Crew

Keep only the mechanisms that enforce the contract



Orchestrator

Owns sequence; agents allow-list; no edit tool



Four roles

One boundary each, and each one held in the run



Worktrees

One per workstream, with budgets on every session

```
.github/agents/sdlc-orchestrator.agent.md · FanHub 69458ee
↳ planner · implementer · test-writer · test-reviewer
```

The Orchestrator Owns Sequence and Delegates Every Edit

.github/agents/sdlc-orchestrator.agent.md

```
---
name: sdlc-orchestrator
description: Runs one FanHub Node frontend issue
  through plan, implement, test, review, and
  integrate using narrow subagents.
tools: ["agent", "read", "search", "execute", "todo"]
agents: ["planner", "implementer",
         "test-writer", "test-reviewer"]
hooks:
  Stop:
  - type: command
    timeoutSec: 300
    command: >-
      node scripts/copilot-hooks/verify-before-stop.mjs
---
```

📋 agents is an allow-list

The orchestrator may call these four and no others

✋ tools omits edit

Every change goes through a role agent

🛑 Agent-scoped Stop hook

The gate runs whenever the orchestrator tries to finish

🔢 Three modes in the body

plan, implement, integrate, with 'two rounds at most, then report'

Each Role Agent Gets One Boundary, and Each One Held

🗺️

planner · read, search

Wrote no files; returned plans to the orchestrator

🛠️

implementer · source only

In #5 it reported 8 failed and left the test fix to test-writer

🧪

test-writer · tests only

Reused from Custom Agents; edited only *.test.js files

🔍

test-reviewer · read-only

Returned request-changes, then accept, with findings

🧠 Subagents start with no conversation history: the orchestrator hands each one the plan, the failure, or the findings.

One Worktree per Workstream, a Budget on Every Session

implement #4 · run in parallel with #5

```
copilot --worktree char-detail `
  --agent sdlc-orchestrator --mode autopilot `
  --max-autopilot-continues 15 `
  --max-ai-credits 150 `
  --allow-all-tools `
  --deny-tool 'shell(git push)' `
  --share .\evidence\impl4.md `
  --log-dir .\evidence\impl4-logs `
  -p "implement #4"
```

🌳 --worktree char-detail

Creates FanHub.worktrees\char-detail on a new branch

💳 Budgets on the command line

Continuation cap and AI-credit cap per session

⛔ Deny wins

Deny rules take precedence over every allow rule

📄 --share and --log-dir

A transcript and logs for every session

Three Setup Details the First Runs Taught

🔐 Permissions

Plan session, first attempt

- **shell(gh:*) missed a compound PowerShell command**
- **Autopilot stopped: no progress at 38.67 of 40**
- **Fix: --allow-all-tools with deny git push**

📦 Fresh worktrees

No node_modules

- **sessionStart hook in .github/hooks/**
- **Links main checkout node_modules**
- **Both worktrees ran Jest on arrival**

Hook input casing

Agent-scoped Stop hook

- **Keys arrive as session_id, cwd, stop_reason**
- **First version read sessionId**
- **Logging input keys found it in one run**

🔎 Log the raw hook input and the permission denials; both made each fix a one-run change.

PART 3

Let Evidence Decide When to Stop

Tests steer the run, a Stop hook backs them up, and budgets hand control to a person

🔮

Predicted, then routed

The plan's risk became 8 failures and a routed fix

🛑

Backstop

11 allow and 0 block in real runs; a staged block shows the path

💳

Budget stop

113.4 of 120 credits, then a person resumed

```
.copilot-loop/hook-log.jsonl
↳ decision · blocksSoFar · Tests: line · seconds
```

Predicted in the Plan, Caught by Tests, Routed to test-writer

❌ Implementer run · suite red

Navigation added
Episodes.js now calls useNavigate()

8 failed, 8 total
useNavigate() may be used only in the context of a <Router>

Boundary held
Implementer reported it; tests are outside its role

💡 Orchestrator routed the failure

To test-writer
Add router context to the existing render helper

Plus new coverage
EpisodeDetail and App route tests

✅ Suite green, reviewer engaged

Reviewer requested revisions, then accepted

Commit 24b6a6c on branch episode-detail

The risk named in Part 1 became a routed fix

47 / 47
tests passing in the #5 worktree

💡 Evidence steered the run: a named risk, a red suite, a routed fix, a green suite.

The Stop Hook Turns Done Into a Test Result

scripts/copilot-hooks/verify-before-stop.mjs (core)

```
// CI=true npx react-scripts test --watchAll=false
const run = spawnSync(testCmd, { cwd: frontend,
  shell: true, env: { ...env, CI: "true" } });
if (run.status === 0) return allow();
if (blocks >= 3) return escalate(); // note, allow
saveBlocks(blocks + 1);
print({ decision: "block",
  reason: `Stop hook (block ${blocks + 1} of 3):
  suite failing.\n${failures}\nRoute to
  implementer or test-writer, rerun, finish.` });
```

⛔ block + reason

The CLI feeds the failing output back as the next prompt

✅ allow

The suite passed, so the orchestrator may finish

3️⃣ Three strikes, then escalate

Agent-scoped Stop hooks also stop forcing after 8 consecutive blocks

📜 Every decision logged

JSONL in gitignored .copilot-loop/, per session

Across Real Runs the Gate Allowed 11 Times and Blocked None

Stop hook decisions · REAL

Plan session
Allow · 8/8 on main

Implement #4 · #5
Allow · 56/56 and 47/47

Integrate
Allow · 96/96

Total
11 allow · 0 block

What caught failures · REAL

The orchestrator's own step
Run the suite, route failures, rerun, then stop

#5 in-loop
8 failed → test-writer → 47/47

Integrate in-loop
Reviewer finding → fix + regression test → 96/96

The orchestrator tests before stopping; the gate covers the run that skips it.

Told to Skip Tests, the Orchestrator Met the Gate

hook-demo/.copilot-loop/hook-log.jsonl · STAGED


```
{"branch":"hook-demo","decision":"block",
 "blocksSoFar":0,
 "tests":"Tests: 7 failed, 1 passed, 8 total",
 "seconds":5}
{"branch":"hook-demo","decision":"allow",
 "blocksSoFar":1,
 "tests":"Tests: 8 passed, 8 total",
 "seconds":4}
```


Setup

A planted commit renamed Season to Series in the episode filter

Prompt

Skip tests and subagents; summarize and finish

Block

7 failed: the reason arrived with the failing assertions

Allow

test-writer updated expectations · 8/8 · commit 33163d2 · 26.89 credits

Tests Prove Agreement; a Person Judges Intent

✅ What the gate proved

Code and tests agree
8 of 8 after the Series rename

Fast and repeatable
About five seconds per decision

Logged per session
Decision, strike count, Tests: line

🤦 What stays with a person

Was the rename wanted?
The agent treated it as intended and changed tests to match

Is the behavior right?
Product intent lives in review

Ready to merge?
The reviewer owns that call

💡 A green gate says the code and its tests agree. Whether the change was wanted is a person's call.

Budgets Stopped Two Runs Cleanly and Handed Control to a Person

Each stop left a clean state, a transcript, and a stated reason

--max-ai-credits	Whole session, subagents included; warnings at 50, 75, 90 percent	`integrate: 113.4 of 120`
--max-autopilot-continues	Autopilot continuation turns; set to 15 for implementation	`cap unused`
No-progress stop	Repeated turns without a successful tool call	`plan: permissions`
Hook strike counter	Three blocked stops, then an escalation note	`unused in real runs`

A person read the summary, clarified the integration record is the PR body, and resumed at 140 credits

PART 4

Integrate, Then Hand It to a Person

Resolve named seams, review the merged whole, and open a draft with its evidence

🔍

Merged-whole review

A defect that existed only after two green branches met

📜

Integration record

Seams, counts, corrections, open risks

🤝

Draft handoff

The reviewer owns merge and intent

```
-p "integrate char-detail episode-detail into feat/node-detail-pages for #4 #5"
↳ draft #217 · Fixes #4 · Fixes #5
```

After Merge, the Reviewer Found a Defect Neither Branch Had

❌ Merged whole · reviewer finding

Both branches green
56/56 and 47/47 before merging

Unreadable headings
Global white heading rules on new white panels

Visible after merge
Found in review of the integrated diff against main

💡 Routed through the roles

implementer
Local heading colors in CharacterDetail.jsx

test-writer
Real-stylesheet contrast regression, 4.5:1 for all five headings

✅ Integrated and recorded

Listed as a reviewer correction in the record

Merge commit 036a070 · branch pushed

Draft #217 opened with Fixes #4 and Fixes #5

95 → 96
tests after the contrast regression, all passing

💡 Review the merged whole: integration is where workstreams meet for the first time.

The Reviewer Receives the Evidence and the Open Risks

Integration record = PR body

Seams
App.js auto-merged; App.test.js kept all 11 route tests

Counts
56 and 47 before · 95 after · 96 with the regression

Corrections
Contrast fix with its regression test

Open risks
Strict CI lint warnings pre-existing; live backend unverified

Who owns what next

Reviewer
Merge decision and the intent behind every change

Team lead
Budget and escalation policy for the next run

Orchestrator
Pushed one branch, opened a draft, and stopped

The loop ends where human authority begins: plan on the issues, evidence in the PR, merge with a person.

Write a Loop Contract for Your Next Issue

🎯 Your loop contract

Roles
Who plans, edits source, edits tests, judges

Done
The fastest meaningful test command your Stop hook runs

Budget
How many credits the issue is worth

Merge
Who reads the draft before it merges

🧭 Where it stops fitting

Event-started work
Labels, schedules, failed checks: Agentic Workflows

Assign and walk away
Cloud agent → Copilot on GitHub

Small changes
The plan stage alone used 47 credits here

Slow verification
A gate that takes minutes changes every blocked stop

❓ Your next issue: which command does the Stop hook run, what is it worth, who reads the draft?

From Agents Handling It to a Loop With a Contract

Before

- One agent asked to do everything in one session
- Done means the agent said it was done
- Runs end when someone notices the spend
- A diff arrives with a summary to trust

After

- An orchestrator with four narrow roles and named seams
- Done means the suite passed and the reviewer accepted
- Credit and continuation caps on every session
- A draft arrives with plan, counts, corrections, and risks

8 failed → 47/47
Predicted risk caught in-loop and routed (REAL)

11 allow · 0 block
Real gate record; staged block → allow shows the backstop

95 → 96
Reviewer catch after merge, locked by a regression test (REAL)

✅ WHAT YOU CAN DO TODAY

Today

- Write the four-field loop contract for one issue in your repository
- Pick the fastest meaningful test command a Stop hook could run
- Decide the AI-credit cap the issue is worth

This Week

- Create an orchestrator with an agents allow-list and no edit tool
- Add a read-only planner that writes risks before code
- Add a Stop hook that logs every decision and stops forcing after three blocks

This Month

- Run two workstreams in parallel worktrees with --share and --log-dir
- Integrate with a written record as the draft PR body
- Agree team budgets and escalation policy with your lead

🔑 Key Takeaway

Roles own the work, tests decide done, budgets decide when to stop, and a person decides what merges.

REFERENCES

Agents and Hooks

- **VS Code: subagents**
  Orchestration patterns
- **Custom agents configuration**
  agents, tools, hooks
- **Custom agents for Copilot CLI**
  CLI setup
- **Hooks reference**
  Stop, decisions, agent scope

Copilot CLI

- **Autopilot mode**
  Continues and credit caps
- **Multiple sessions**
  Worktrees and parallel work
- **CLI command reference**
  Flags and permissions
- **Delegating and parallelizing**
  Choosing agent work

FanHub Evidence

- **sdlc-orchestrator.agent.md**
  The orchestrator
- **verify-before-stop.mjs**
  The Stop hook
- **#4 plan**

Related Talks

- **Custom Agents**
  One agent file, every runtime
- **Copilot CLI**
  The terminal surface this loop runs in
- **Copilot Hooks**



Local Agentic SDLC

A LOOP WITH A CONTRACT, ROLES, EVIDENCE, BUDGETS, AND A PERSON AT THE MERGE

8 failed → 47/47

The plan predicted it; the orchestrator routed it

95 → 96

A reviewer catch that existed only after merge

≈380 credits

Two issues to one integrated draft, one observed run

Which command would your Stop hook run on your next issue, and who reads the draft before it merges?